

Report Development Guide

Table of Contents

1. Prerequisites & Environment Setup

2. Project Structure Overview

[2.1 Creating the Report Folder](#)

[2.2 Sub-folder Structure: Model, SQL, Report](#)

3. SQL Layer

[3.1 Writing the SQL Query](#)

[3.2 Creating the .resx Resource File](#)

[3.3 Adding SQL Strings to the Resource File](#)

[3.4 Accessing the Resource File from C#](#)

4. Model Layer

[4.1 What is a Model Class?](#)

[4.2 Creating Model.cs](#)

[4.3 Mapping SQL Columns to Model Properties](#)

5. Report Layer (C#)

[5.1 Creating the Report C# File](#)

[5.2 XrmReport Class Structure](#)

[5.3 Defining Parameters \(Filters\)](#)

[5.4 RunReport Method](#)

[5.5 Maintaining Consistency with Existing Reports](#)

[5.6 Resolving Dependencies & Errors](#)

[5.7 Clean, Build & Rebuild](#)

6. RDLC Design (Microsoft Report Designer)

[6.1 Adding the RDLC File to the Project](#)

[6.2 Report Designer Overview](#)

[6.3 Adding a Dataset](#)

[6.4 Designing the Report Layout](#)

[6.5 Adding a Table / Tablix](#)

[6.6 Binding Fields to Columns](#)

7. Deployment to Ezra360

[7.1 Build in Release Mode](#)

[7.2 Config Section – Uploading the DLL \(Assemblies\)](#)

[7.3 Config Section – Uploading the RDLC \(Report\)](#)

[7.4 Testing on DEV](#)

[7.5 Deploying to UAT](#)

1. Prerequisites & Environment Setup

Setting up your development Environment

First, install **Visual Studio (Community edition)** and include the **.NET desktop development** workload (and optionally ASP.NET/web development).

Make sure the **.NET Framework 4.7.2 targeting pack** is selected.

After installation, sign in and confirm everything runs properly.

Next, install key extensions in Visual Studio:

- **Microsoft RDLC Report Designer (required)** for working with report files
- Productivity Power Tools and GitLens (recommended for improved workflow and Git tracking)

Then set up **Git**:

- Install Git with default settings
- Configure your username and email
- Clone the EzraReports repository using the provided URL and open the solution in Visual Studio

Finally, add required **SDK references** to your project:

- Crm.SoftAer.Com.Model.dll (data models and enums)
- Xrm.Soft_Aer.SDK.dll (report interfaces and execution context)

These DLLs must be manually added via *Add Reference* → *Browse* from the shared libraries folder (usually /libs or /dependencies).

2. Project Structure Overview

2.1 Creating the Report Folder

Each report is built as its own **Class Library (.NET Framework)** project. Follow the naming convention used by existing reports in the solution.

1 Add a new project

In Visual Studio, right-click the Solution in **Solution Explorer** (the panel on the right side) → **Add** → **New Project**.

Search for **Class Library (.NET Framework)** and select it. Name it using this convention:

```
ERP.Ezra360.YourReportName
```

2 Delete the default class

Visual Studio creates a default **Class1.cs** file. Right-click it in Solution Explorer and delete it — you will create your own files.

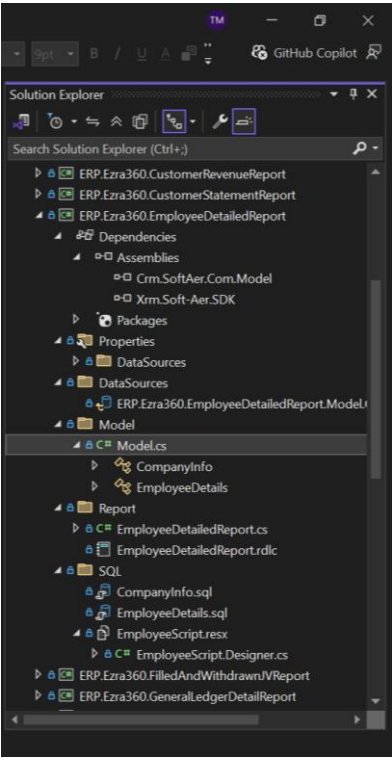
2.2 Sub-folder Structure: Model, SQL, Report

Inside the project, create three sub-folders. Right-click the project → **Add** → **New Folder** and name each one:

- **Model** — holds the C# classes that represent the shape of your query data
- **SQL** — holds the .resx resource file containing all SQL query strings
- **Report** — holds the main C# report class and the .rdlc layout file

Your project structure should look like this:

```
ERP.Ezra360.YourReportName/  
  ■■■ Model/  
    ■ ■■■ YourReportModel.cs  
  ■■■ SQL/  
    ■ ■■■ SQLResources.resx  
    ■ ■■■ SQLResources.Designer.cs (auto-generated)  
  ■■■ Report/  
    ■■■ YourReport.cs  
    ■■■ YourReport.rdlc
```



3. SQL Layer

The SQL layer is responsible for retrieving data from the Ezra360 database. You write your queries in SQL, store them in a resource file, and then reference them from the C# report class.

3.1 Writing the SQL Query

Before creating any files, write and verify your SQL query. The query defines exactly which columns of data your report will display.

Example of *EmployeeDetails.sql*

```
SELECT
    E.Name,
    E.EmployeeNo,
    E.NationalityIdName AS Nationality,
    TT.OptionText      AS Ethnicity,
    PT.OptionText      AS Gender,
    E.Email,
    E.Mobile,
    E.DepartmentIdName AS Department,
    E.StartDate,
    PVT.OptionText     AS EmployeeStatus
FROM Employee E
LEFT JOIN Department D ON D.DepartmentId = E.DepartmentId
LEFT JOIN Picklist GenderList ON GenderList.Uniquename = 'Gender' LEFT JOIN
PicklistItem PT ON PT.PicklistId = GenderList.PickListId
AND PT.OptionValue = E.GenderId
LEFT JOIN Picklist EthnicityList ON EthnicityList.Uniquename = 'Ethnicity' LEFT JOIN
PicklistItem TT ON TT.PicklistId = EthnicityList.PickListId
AND TT.OptionValue = E.EthnicityId
LEFT JOIN Picklist EmpStatusList ON EmpStatusList.Uniquename = 'EmployeeStatus'
LEFT JOIN PicklistItem PVT ON PVT.PicklistId = EmpStatusList.PickListId
AND PVT.OptionValue = E.EmployeeStatusId
WHERE E.IsActive = 1
```

3.2 Creating the .resx Resource File

A **.resx** (Resource) file is an XML file that stores key-value pairs. In this project, it is used to store SQL query strings as named resources, keeping them organised and separate from the C# logic.

1 Add the resource file

Right-click the **SQL** folder in Solution Explorer → **Add** → **New Item**.

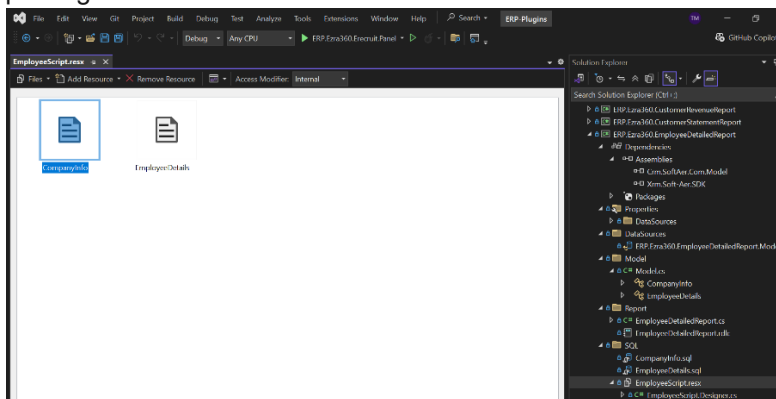
In the search box, type **Resources** and select **Resources File**. Name it **SQLResources.resx** and click Add.

2 Verify auto-generated Designer file

Visual Studio automatically creates a **SQLResources.Designer.cs** file alongside the .resx. Do not edit this file manually — it is regenerated automatically.

3.3 Adding SQL Strings to the Resource File

With the .resx file open, you can drag and drop SQL queries directly into the .resx file instead of manually pasting them.



■ Give each SQL entry a clear, descriptive name that matches the dataset it returns. This makes the resource file easy to navigate as more reports are added.

3.5 Accessing the Resource File from C#

Once the .resx file is saved, Visual Studio generates a strongly-typed class from the Designer file. You access the SQL strings like this:

```
// Access a SQL string by its resource name
string sql = SQLResources.EmployeeDetails;

// Pass it to the Ezra360 query context
var results = context.RetrieveFromXql(SQLResources.EmployeeDetails, "EmployeeDetails");
```

4. Model Layer

4.1 What is a Model Class?

A model class is a simple C# class whose properties represent the columns returned by your SQL query. When Ezra360 executes your query, it maps each row of results into an instance of your model class. Think of it as a template that matches the shape of your data.

■ ■ The property names in your model class must exactly match the column aliases in your SQL SELECT. If your SQL says 'E.DepartmentIdName AS Department', your model must have a property called 'Department'.

4.2 Creating Model.cs

1 Add the file

Right-click the **Model** folder → **Add** → **New Item** → **Class**. Name it **Model.cs**

2 Define the namespace

Set the namespace to match the project and folder:

```
namespace ERP.Ezra360.EmployeeDetailedReport.Model
```

For each column alias in your SQL SELECT, add a matching property to the model class. Use **string** for most fields. Use **DateTime?** for date fields if needed.

```

using System;
using System.Collections.Generic;
using System.Text;

namespace ERP.Ezra360.EmployeeDetailedReport.Model
{
    public class CompanyInfo
    {
        public string CompanyName { get; set; }
        public string Email { get; set; }
        public string Telephone { get; set; }
        public string WebAddress { get; set; }
        public string VATNo { get; set; }
        public string CompanyAddress { get; set; }
    }

    public class EmployeeDetails
    {
        public string Name { get; set; }
        public string EmployeeNo { get; set; }
        public string Nationality { get; set; }
        public string Ethnicity { get; set; }
        public string IDType { get; set; }
        public string IdNumber { get; set; }
        public string Gender { get; set; }
        public string DisabledId { get; set; }
        public string MaritalStatus { get; set; }
        public string HomeLanguage { get; set; }
        public string Email { get; set; }
        public string Mobile { get; set; }
        public string Role { get; set; }
        public string Department { get; set; }
        public string EmployeeGroupIdName { get;
set; }
        public string DirectReportIdName { get;
set; }
        public string NatureofContract { get; set; }
    }

    public string WorkModel { get; set; }
    public string PayrollCycleIdName { get;
set; }
    public string StartDate { get; set; }
    public string EmployeeStatusId { get; set; }
    }

    public string TerminationDate { get; set; }
    }

    public string TaxRefNo { get; set; }
    public string PAYEAmount { get; set; }
    public string DelegatedAuthorityName {
get; set; }
    }
}

```

■ ■ A mismatch between a SQL alias and a model property name means that field will always be null/blank in the report — no error will be thrown, the data simply won't appear.

5. Report Layer (C#)

The Report class is the core of the report. It tells Ezra360 what parameters (filters) to show the user, and how to fetch and prepare the data when the report runs.

5.1 Creating the Report C# File

1 Add the file

Right-click the **Report** folder → **Add** → **New Item** → **Class**. Name it **EmployeeDetailedReport.cs**

2 Add using statements

At the top of the file, add these using statements:

```
using Crm.SoftAer.Com.Model;
using ERP.Ezra360.YourReportName.Model;
using ERP.Ezra360.YourReportName.SQL;
using System.Collections.Generic;
using System.Linq;
using Xrm.SoftAer.SDK;
```

This class defines the structure of a report filter parameter. Copy this class exactly into every report — it does not change between reports.

```
public class YourReportParameter : IReportParameter
{
    public string DisplayMember { get; set; }
    public string UniqueName { get; set; }
    public object Value { get; set; }
    public object Data { get; set; }
    public bool EnableMutiselect { get; set; }
    public DataType ParameterType { get; set; }
    public int SortIndex { get; set; }
    public Dictionary<string, object> LocalListData { get; set; }
    public string SchemaName { get; set; }
}
```

5.2 IXrmReport Class Structure

Your main report class implements **IXrmReport**. It must have three properties and two methods as shown below:

```
public class YourReport : IXrmReport
{
    public YourReport() { /* constructor - set up parameters here */ }

    public List<IReportParameter> Parameters { get; set; }
    public Dictionary<string, object> ReportDataSources { get; set; }
    public string ReportRDL { get; set; }

    public void ParameterChanged(IRepoExecutionContext context,
        IReportParameter parameter) { }

    public void RunReport(IRepoExecutionContext context) { /* fetch data */ }
}
```

5.3 Defining Parameters (Filters)

Parameters are the filter controls that appear on the report screen before the user runs the report. They are added in the constructor. If no filters are needed, leave the list empty.

Data Type	What it shows the user	SchemaName required?
DataType.Lookup	A searchable lookup field (e.g. choose an Employee or Department)	Yes - entity name
DataType.LocalList	A static dropdown with manually defined options	No
DataType.SelectOption	A dropdown populated from a Picklist in the system	Yes - picklist name

Example — adding a Lookup parameter:

```
public YourReport()
{
    Parameters = new List<IReportParameter>();

    Parameters.Add(new YourReportParameter()
    {
        DisplayMember = "Department", // Label shown to the user
        UniqueName = "DepartmentId", // Key used to retrieve the value in RunReport
        ParameterType = DataType.Lookup,
        SchemaName = "Department" // Entity name in Ezra360
    });
}
```

■■ Never initialise `Parameters = new List()` more than once in the constructor. A second initialisation wipes all previously added parameters, causing a `NullReferenceException` at runtime.

Example — no filters (return all records):

```
public YourReport()
{
    Parameters = new List<IReportParameter>(); // No
    parameters added - report returns all active records
}
```

5.4 RunReport Method

The RunReport method is called when the user clicks Run on the report screen. It fetches data from the database using the SQL strings from the resource file and maps the results into the model classes.

```
public void RunReport(IRepoExecutionContext context)

{

    ReportDataSources = new Dictionary<string, object>();

    var employeeDetailsBE =
context.RetrieveFromXql(EmployeeScript.EmployeeDetails,
"EmployeeDetails");

    var employeeDetail = new List<EmployeeDetails>();

    foreach (var be in employeeDetailsBE)

    {

        var employeeDetails = be.ToEntity<EmployeeDetails>();

        employeeDetail.Add(employeeDetails);

    }

    ReportDataSources.Add("EmployeeDetailsDS", employeeDetail);

    var companyInfoBE =
context.RetrieveFromXql(EmployeeScript.CompanyInfo, "CompanyInfo");

    var companyInfo = new List<CompanyInfo>();

    foreach (var be in companyInfoBE)

    {

        var companyInformation = be.ToEntity<CompanyInfo>();

        companyInfo.Add(companyInformation);

    }

    ReportDataSources.Add("CompanyInfoDS", companyInfo);

}
```

■ ■ The string passed to `ReportDataSources.Add()` (e.g. 'EmployeeDetailsDS') must exactly match the `DataSet` name you configure in the RDLC designer. This is how the report layout knows where to pull its data from.

5.5 Maintaining Consistency with Existing Reports

When building a new report, always open an existing, working report in the solution and use it as a reference. Specifically:

- Use the same namespace convention: **ERP.Ezra360.YourReportName.Report**

- Copy the **IReportParameter** class exactly — it should never be modified
- Follow the same order of operations in RunReport: fetch data → map to model → add to DataSources
- Keep the same naming pattern for DataSource keys (e.g. EmployeeDetailsDS, CompanyInfoDS)

5.6 Resolving Dependencies & Errors

After writing the code, Visual Studio may show red underlines (errors). These are usually caused by missing references or incorrect namespaces.

Error	Fix
'IXrmReport' could not be found	Add reference to Xrm.Soft_Aer.SDK.dll
'DataType' could not be found	Add reference to Crm.SoftAer.Com.Model.dll
Namespace not found	Check the using statements at the top of the file
'SQLResources' does not exist	Ensure the .resx file is in the SQL folder and Access Modifier is set
Model property always null in report	Verify the SQL alias matches the C# property name exactly

5.7 Clean, Build & Rebuild

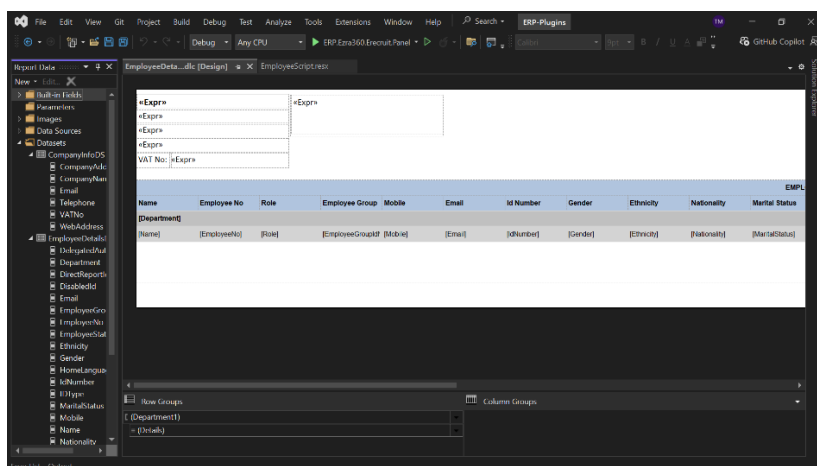
Once all errors are resolved, follow this sequence before moving to RDLC design:

- Start by **cleaning the solution** to remove old compiled files.
- Then **build the solution** and confirm it says *Build succeeded* in the Output window.
- If any issues appear, run **Rebuild Solution**, which does a full clean and build in one step.

■■ **Never proceed to RDLC design or deployment while there are build errors. All errors in the Output panel must be resolved first.**

6. RDLC Design (Microsoft Report Designer)

The RDLC file defines how the report looks - the layout, columns, formatting, headers and expressions. It is designed visually inside Visual Studio using the Microsoft Report Designer.



6.1 Adding the RDLC File to the Project

1 Add a new Report file

Right-click the **Report** folder in Solution Explorer → **Add** → **New Item**.

In the search box, type **Report** and select **Report** (this creates an .rdlc file). Name it to match your report class, e.g. **EmployeeDetailedReport.rdlc**.

2 Open the designer

Double-click the .rdlc file in Solution Explorer. The Microsoft Report Designer opens in the main editor area.

6.2 Report Designer Overview

When the Report Designer is open, you will see the following panels:

Panel	Location	Purpose
Report Data	Left side panel	Where you add Datasets and Parameters that feed data into the report
Design canvas	Centre (main area)	Where you drag and drop report elements like tables, text boxes, images
Properties	Right side panel	Where you format selected elements — font, colour, size, borders
Toolbox	Left side (tabbed)	Contains the elements you can add: Table, Text Box, Image, Line,

6.3 Adding a Dataset

A Dataset connects the RDLC to the data provided by the C# RunReport method. You must add one Dataset per DataSource you added in RunReport.

1 Open Report Data panel

On the left side, click the **Report Data** tab. You will see folders for Built-in Fields, Parameters, Images, and Data Sources.

2 Add a DataSet

Right-click **Datasets** → **Add Dataset**.

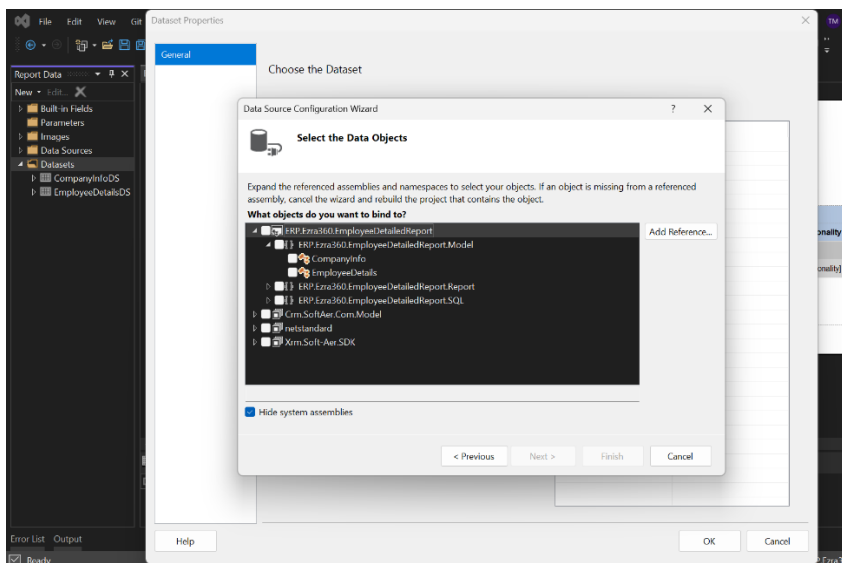
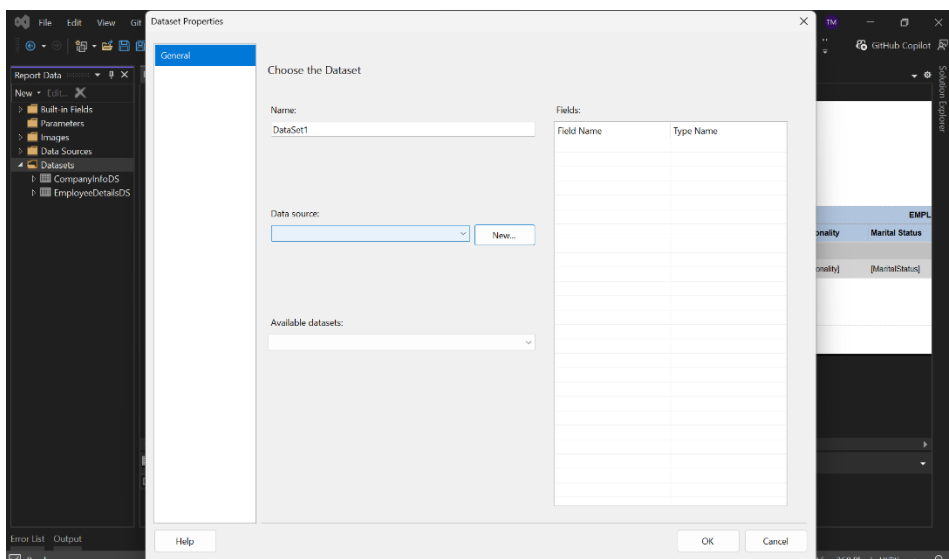
In the dialog that opens:

- Set **Name** to match exactly what you used in ReportDataSources.Add() - e.g. **EmployeeDetailsDS**
- Select **Use a dataset embedded in my report**
- In **data source type**, select **Object**

3 Add fields to the Dataset

After the Dataset is created, right-click it → **Add** → **Field**. Add one field per property in your model class. The field names must match the model property names exactly.

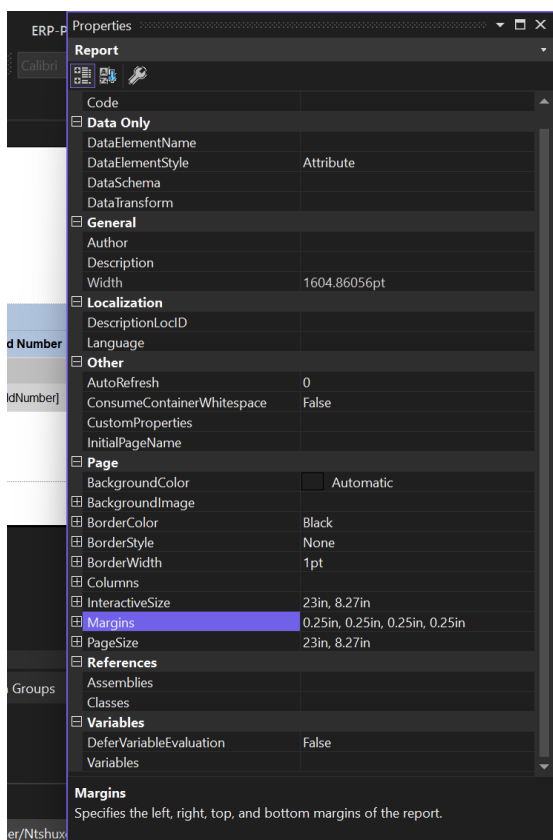
■ ■ The Dataset name in the RDLC must exactly match the key used in `ReportDataSources.Add()` in your C# code. A mismatch means no data will appear in the report.



6.4 Designing the Report Layout

The design canvas is a blank area where you build the visual layout of your report. It works similarly to a Word document - you can add sections, tables, and text boxes.

- The canvas has a **Header**, **Body** and **Footer** section
- Drag elements from the **Toolbox** onto the canvas
- Click any element to select it and edit it via the **Properties** panel on the right
- Resize elements by dragging their edges



6.5 Adding a Table / Tablix

A Tablix (Table) is the main element used to display rows of data from your Dataset.

1 Insert a Table

In the **Toolbox**, find **Table** and drag it onto the report canvas body.

A basic table with a header row and a data row is inserted automatically.

2 Set the Dataset for the Table

Click anywhere on the table to select it. In the **Properties** panel, find the **DataSetName** property and set it to **EmployeeDetailsDS**.

6.6 Binding Fields to Columns

Each column in the table needs to be linked to a field from the Dataset.

Bind the data cell

Click the bottom cell (the data row) of each column.

Click the small square that appears on the right of the cell → select **Expression** or simply type the field binding:

```
=Fields!Name.Value
=Fields!Department.Value
=Fields!Email.Value
```

Alternatively, click the bottom cell (data row) of each column. When you hover over the cell, a dropdown arrow appears on the right - click it to see a list of all available fields from your Dataset. Select the field you want to bind directly from the list.

Expressions allow you to format, combine, or conditionally display data. They always start with an equals sign (=).

Use Case	Expression Example
Display a field value	=Fields!Name.Value
Combine first and last name	=Fields!FirstName.Value & " " & Fields!LastName.Value
Format a date	=Format(Fields!StartDate.Value, "dd MMM yyyy")
Show blank if null	=IIF(IsNothing(Fields!Mobile.Value), "", Fields!Mobile.Value)
Page Number	="Page " & Globals!PageNumber & " of " & Globals!TotalPages

7. Deployment to Ezra360

Deployment involves building the project in Release mode and uploading two files to the Ezra360 Config section: the compiled **DLL** (the C# code) and the **RDLC** (the report layout).

7.1 Build in Release Mode

1 Build the Solution

Go to **Build** → **Build Solution** (Ctrl+Shift+B). Confirm the Output panel shows **Build succeeded**.

2 Locate the DLL

Navigate to your project's output folder:

```
ERP.Ezra360.YourReportName/bin/Release/ERP.Ezra360.YourReportName.dll
```

This is the file you will upload to Ezra360 as the Assembly.

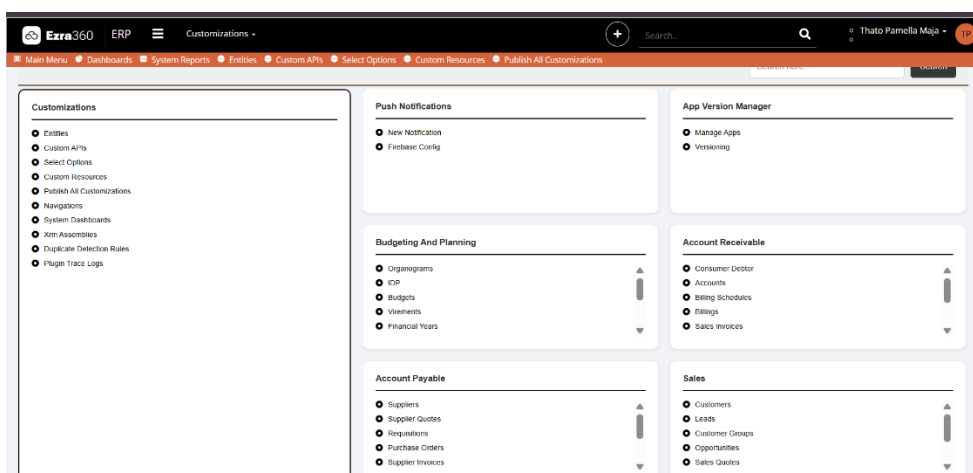
7.2 Config Section – Uploading the DLL (Assemblies)

1 Open the Config section

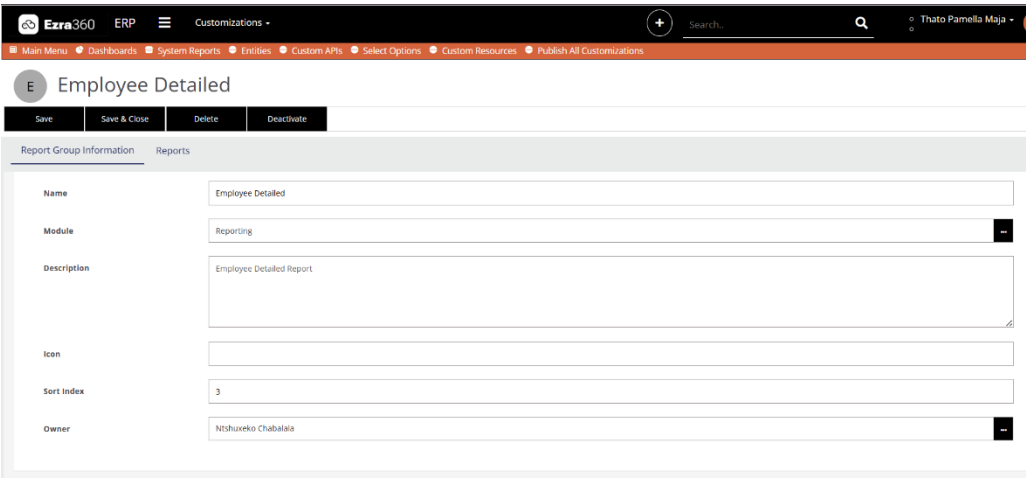
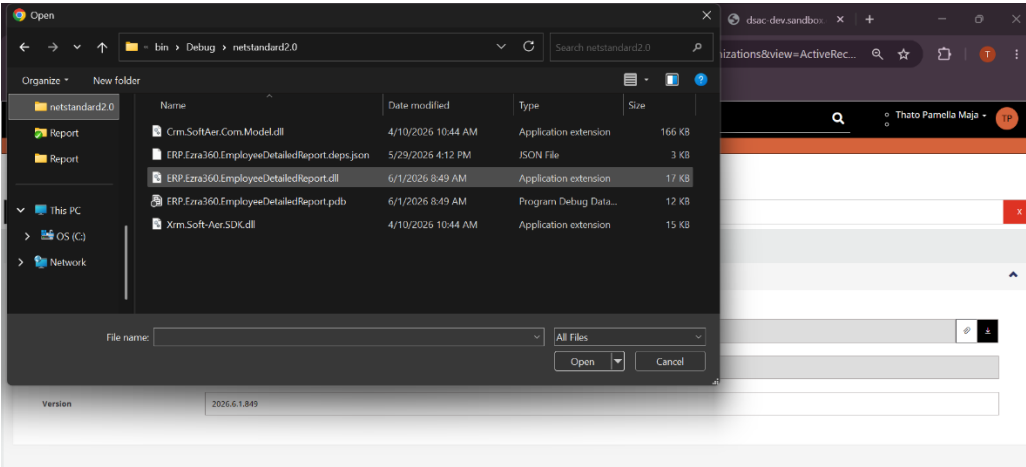
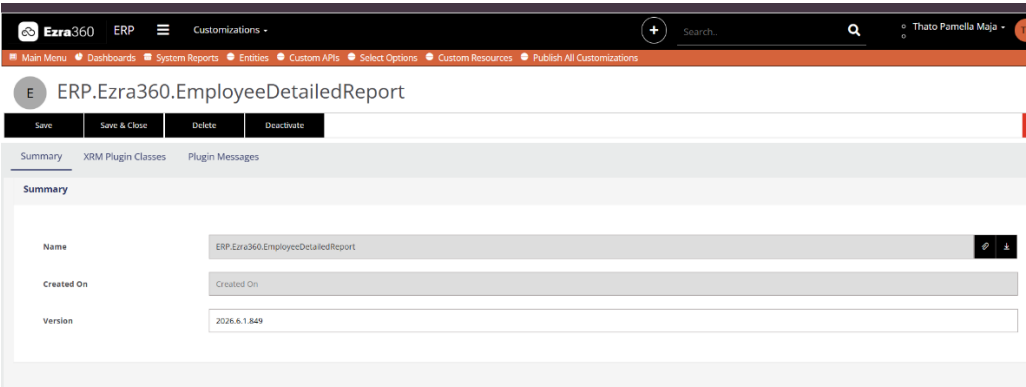
Log into the Ezra360 environment (DEV or UAT). Navigate to the **Config** section in the admin area.

2 Go to Assemblies

- Inside Config, find the **Xrm Assemblies** option. This is where all report DLLs are managed.
- Upload the DLL.
- If you are updating an existing report, find the existing assembly record and upload the new DLL to replace it.



Active Report Groups						
New Delete Deactivate Import Export						
☐	Name	Created On	Created By	Modified On	Modified By	
☐	Age Analysis	2025-11-11 16:01:46	Ntshuxeko Chabalala	2025-12-02 12:33:01	Ntshuxeko Chabalala	
☐	Case Summary	2026-01-20 08:39:02	Ntshuxeko Chabalala	2026-01-20 08:39:02	Ntshuxeko Chabalala	
☒	Employee Detailed	2026-05-22 12:33:47	Ntshuxeko Chabalala	2026-05-22 12:33:47	Ntshuxeko Chabalala	
☐	General Ledger	2025-12-04 23:01:13	Ntshuxeko Chabalala	2025-12-05 00:29:34	Ntshuxeko Chabalala	Employee Detailed
☐	Leave Tracking Report	2025-09-29 09:08:10	Ntshuxeko Chabalala	2025-11-20 11:57:11	Ntshuxeko Chabalala	
☐	Revenue Analysis	2025-11-10 16:10:49	Ntshuxeko Chabalala	2025-12-02 12:48:30	Ntshuxeko Chabalala	
☐	Sales Template	2026-05-24 12:02:48	Ntshuxeko Chabalala	2026-05-27 16:45:38	Ntshuxeko Chabalala	
☐	Statements	2025-12-02 09:05:16	Ntshuxeko Chabalala	2025-12-02 09:05:16	Ntshuxeko Chabalala	
☐	Timesheet	2026-05-18 14:28:40	Ntshuxeko Chabalala	2026-05-18 14:28:40	Ntshuxeko Chabalala	
☐	VAT 201 Report	2025-10-13 09:41:07	Ntshuxeko Chabalala	2025-10-14 16:16:23	Ntshuxeko Chabalala	



7.3 Config Section – Uploading the RDLC (Report)

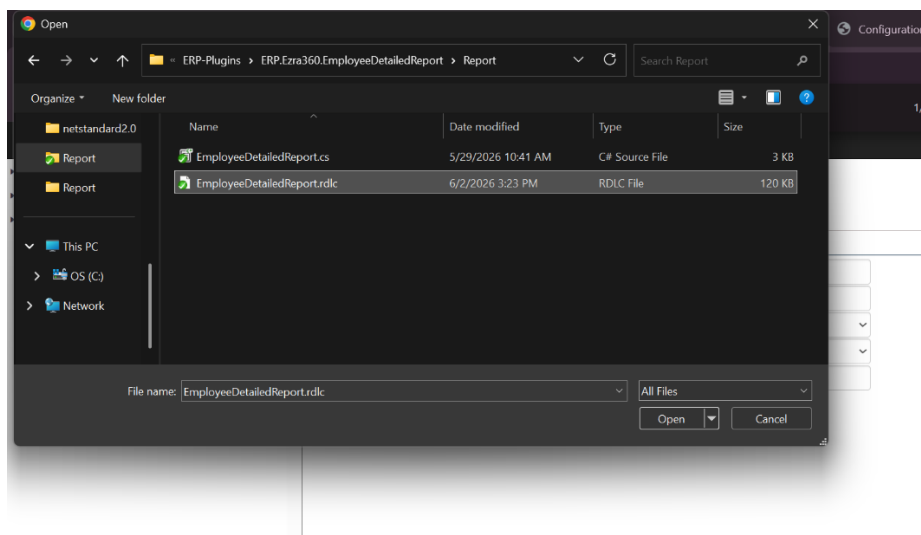
1 Go to Reports

Inside Config, find the **Reports** section. This is where report records are created and managed.

2 Create or update the report record

Click **Add New Report** (or open the existing record to update it) and fill in the following fields:

- **Report Name** — the display name shown to users in the BI Portal
- **Assembly** — select the DLL you just uploaded
- **Report Class** — the fully qualified class name, e.g.
ERP.Ezra360.YourReportName.Report.YourReport
- **RDLC File** — upload the .rdlc file from your Report folder

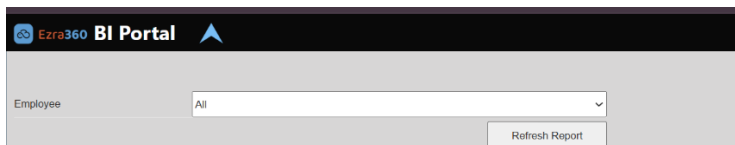
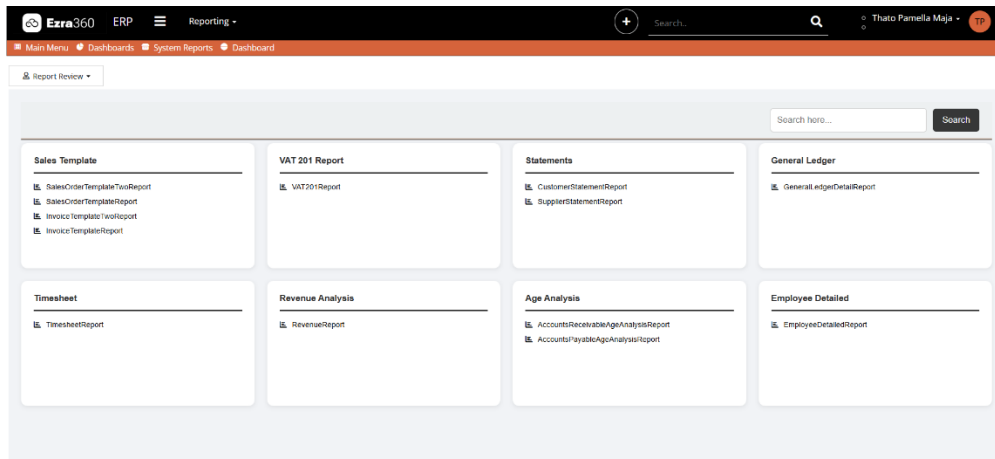


NB: Publish after each deployment

7.4 Testing on DEV

After deployment to DEV, open the Portal and run the report.

- Verify the report loads without errors
- Confirm data is returning correctly - check a few employee records manually
- Check for duplicate rows - if found, review the PicklistItem JOIN pattern
- Verify all columns are populated where expected
- Test any parameters/filters if applicable



EMPLOYEE DETAILS														
Name	Employee No	Role	Employee Group	Mobile	Email	Id Number	Gender	Ethnicity	Nationality	Marital Status	Home Language	Disabled	Direct Report To	Work Model
Vice Mkwani	EMP-00075			0815108347	vicemkwani@gmail.com	0401016763187							Siyanda	
Moses Sebetha	EMP-00085	Software Tester/Quality Assurance	Software Testers	0876533556	mosebutha@gmail.com	234567897654322345		African						Part Time
Norokuleko Guma	EMP-00019			0857201724	norokuleko@gmail.com	0401012628186	Female						Derehl Maswanganyi	
Mogau Santili	EMP-00106	Account Manager	Developers	0691548169	mogau.santili@xiquelgroup.com	9606016582196	Male	African						Part Time
Mogau Santili	EMP-00107	Account Manager	Developers	0691548169	mogau.santili@xiquelgroup.com	9606016582196	Male	African					Mogau Santili	Part Time
Mogau Santili	EMP-00107	Account Manager	Developers	0691548169	mogau.santili@xiquelgroup.com	9606016582196	Male	African						Part Time
Derehl Maswanganyi	EMP-00049			+27787979994	DerehlMaswanganyi@gmail.com	9001017262588	Male	African		Single	English		Derehl Maswanganyi	
Siyanda Zibikha	EMP-00077			+27818362271	siyandashiba@gmail.com	0401017687088	Male	African		Single	isiXhosa		Siyanda	
Test User	EMP-00070			+27782827272	ecramel@xiquelgroup.com	0401017652181	Male	African		Single	English		Kaone Emmanuel Senoelo	
Nokuthando Zizane	EMP-00095			0823454040	nokuthandoz@xiquelgroup.com	0201015791182	Female	African		Single	Seutho			

7.5 Deploying to UAT

Once the report is verified on DEV, repeat the exact same upload steps on the UAT environment.

- Log into the UAT environment and navigate to Config
- Upload the same DLL and RDLC files
- Manually check the Employee form on UAT to confirm all fields exist
- If any fields are missing on UAT, report them before testing
- Run the report on UAT and verify results